

Resolution Algorithm – Project Report

Abstract

This project implements the propositional Resolution Algorithm to prove five simple knowledge-base goals. Each problem is represented using propositional logic, converted into Conjunctive Normal Form (CNF), combined with the negation of the goal, and solved by repeated resolution. A proof is obtained when the Empty Clause (■) is generated.

1. Introduction

Resolution is an inference rule used in automated reasoning. In propositional logic, a goal can be proved by contradiction: the knowledge base is combined with the negation of the goal. If repeated resolution produces the Empty Clause, the combined set of clauses is inconsistent, which means the original goal follows from the knowledge base.

2. Problem Statement

The project considers five rule-based scenarios: Rain and Wet Ground, Student Assignment Submission, Library Membership, Placement Eligibility, and Access Control. For every scenario, the objective is to determine whether the stated goal logically follows from the given facts and rules using resolution.

3. Objectives

- Represent natural-language rules and facts as propositional logic.
- Convert implications into CNF clauses.
- Negate each target goal.
- Implement the Resolution Algorithm in Python.
- Derive the Empty Clause when the goal is logically entailed.
- Display the proof steps and final results.

4. Methodology

Step 1: Define propositional symbols.

Step 2: Translate implications using $P \rightarrow Q \equiv \neg P \vee Q$.

Step 3: Build the CNF clause set.

Step 4: Add the negation of the goal.

Step 5: Select pairs of clauses containing complementary literals.

Step 6: Generate resolvents.

Step 7: Stop when ■ is generated or when no new clauses can be produced.

5. Algorithm

Input: A set of CNF clauses and a goal literal.

1. Add the negation of the goal to the clause set.
2. For each pair of clauses, identify complementary literals.
3. Remove the complementary pair and combine the remaining literals.
4. Add every new resolvent to the clause set.
5. If an empty clause is generated, return PROVED.
6. If no new clauses can be generated, return NOT PROVED.

Output: Proof status and resolution steps.

6. Implementation

The Python implementation represents positive literals as positive integers and negative literals as negative integers. For example, P is represented by 1 and $\neg P$ by -1. A clause is stored as a frozenset of literals. The `resolve()` function generates resolvents, while `resolution_prove()` performs the iterative proof search.

7. Results

All five supplied goals are proved successfully.

Question 1: Ground is wet — PROVED.

Question 2: Rahul receives internal marks — PROVED.

Question 3: Priya can borrow books — PROVED.

Question 4: Arun is eligible for placement — PROVED.

Question 5: User is granted access — PROVED.

In every case, the Python implementation derives the Empty Clause (■).

8. Advantages

- Simple and systematic inference mechanism.
- Suitable for automated theorem proving.
- Works directly with CNF clauses.
- Provides a formal proof by contradiction.
- Can be extended to larger propositional knowledge bases.

9. Limitations

The examples use propositional logic, so each statement is treated as a complete proposition. Large clause sets may cause many pairwise resolutions and can require substantial computation. Real-world systems may need predicate logic, heuristics, indexing, or more advanced SAT-solving techniques.

10. Conclusion

The project demonstrates how the Resolution Algorithm proves logical conclusions from a knowledge base. By converting implications to CNF and adding the negated goal, each of the five scenarios produces an Empty Clause. Therefore, all five goals are logically entailed by their respective knowledge bases.

11. Future Enhancements

The implementation can be extended with predicate logic, resolution-tree visualization, clause-learning techniques, heuristic clause selection, interactive user input, and a graphical interface for entering rules and goals.